Infosys Springboard Virtual Internship 6.0 Completion Report

Team Details <Do not mention any personally identifiable information like email ID, institute details, mobile phone number etc.>

Batch Number 13 : WellBot Global Wellness Assistant Chatbot

Start date : 02-02-2026

Name: Rashi Rathore

Internship Duration: 40-day (12 Weeks)

1. Project Title

WellBot Global Wellness Assistant Chatbot

2. Project Objective

WellBot is an AI-powered wellness assistant chatbot designed to help users monitor and improve their mental health, stress levels, productivity, and daily wellness habits. The main objectives of this project are:

- To provide users with an interactive AI chatbot for wellness support.
- To analyze user lifestyle data and provide insights into their wellness patterns.
- To predict mood, stress levels, and productivity using machine learning models.
- To provide personalized wellness suggestions to improve mental and physical health.
- To create a dashboard that visualizes user wellness analytics in an easy-to-understand format.

This project aims to combine artificial intelligence and data analytics to promote better health awareness and wellness management.

3. Project description in detail

WellBot is a web-based AI wellness platform developed using Python Flask. The platform allows users to track their daily activities, analyze wellness data, and receive intelligent recommendations.

The system consists of multiple modules that work together to provide a complete wellness monitoring experience.

Landing Page:

The landing page introduces users to the platform and highlights the key features of WellBot. It provides information about the purpose of the platform and encourages users to register or log in.

Authentication System:

The system includes a secure login and registration mechanism that allows users to access their personal wellness dashboard.

Wellness Dashboard:

The dashboard is the main interface where users can view their wellness insights. It displays charts and statistics about sleep, mood, work hours, and productivity.

AI Chatbot:

The chatbot interacts with users and provides wellness advice based on their inputs. It acts as a virtual assistant that can guide users in maintaining a healthy lifestyle.

Machine Learning Model;

The platform uses machine learning models built using Scikit-learn to predict mood, stress levels, and productivity. The model is trained on a wellness dataset that includes features such as:

- Sleep hours
- Work hours
- Mood ratings
- Stress level
- Productivity score

The predictions help users understand their mental and emotional health patterns.

Admin Panel:

An admin panel allows administrators to monitor system activity, manage users, and review wellness analytics.

The entire system is designed to provide a simple, user-friendly interface while using advanced technology in the backend.

4. Timeline Overview

Week	Activities Planned	Activities Completed
Week 1	Project objective is decided, making github repository	Github repository created, finalising teams with the project overview
Week 2	Tech stack evaluation, architecture planning	Flask setup completed , learning about flask and fast API's
Week 3	Setup user authentication, JWT tokens	Done with verified signin | login with JWT sessions
Week 4	Learning about API's , setting api's for project , database design and SQL commands	API's is setup using openAi , gemini etc. which makes our project more effective
Week 5	NLP pipeline development – intent detection, sentiment analysis	Keyword extraction and sentiment analysis using NLTK and SpaCy
Week 6	Python functions, dictionary , strings	Implementing python function with the programs
Week 7	GPT models, groq models	Used it in Wellbot
Week 8	Stemming, lemmatization, POS tagging	Used stemming, lemmatization, sentiment analysis of users with their mood tracking
Week 9	Skills included machine learning commands	Displayed Datasets and relevant informations of user based on datasets

Week 10	Dark/Light theme switch, UI polishing	Theme toggle working across UI
Week 11	Dashboard design(like users activity, BMI , profile, feedback)	Implementing dashboard with different languages
Week 12	Adding different chats panels to improve the functionality	Project ready for evaluation

5a. Key Milestones

Milestone	Description	Date Achieved
Project Kickoff	Defined the project objectives, scope, system architecture, and responsibilities for developing the WellBot wellness platform.	**17 Feb 2026**
Prototype / First Draft	Developed the initial working prototype including the Flask backend, basic UI pages, and wellness data input functionality.	**23 Feb 2026**
Mid-Term Review	Evaluated project progress, improved dashboard design, integrated machine learning model, and refined chatbot functionality.	**17 Feb 2026**
Final Submission	Completed the fully functional WellBot system including dashboard analytics, AI chatbot, ML predictions	**23 Feb 2026**
Final Presentation	Demonstrated the working system highlighting features such as wellness tracking, predictions, chatbot interaction, and system architecture.	**28 Feb 2026**

5b. Project execution details

1. Requirement Analysis:
The project started with identifying the main objectives of the WellBot system, which included building an AI-powered wellness assistant that can track user wellness data, provide chatbot support, and generate predictions related to mood, stress, and productivity.
2. System Architecture Design:
The architecture of the system was planned using a client-server model where the frontend interacts with the Flask backend. The backend processes user requests, communicates with the database, and integrates the machine learning model.
3. Frontend Development:
The user interface was developed using HTML, CSS, and JavaScript. Multiple pages were created including the landing page, login page, dashboard page, and chatbot interface to provide a smooth and user-friendly experience.
4. Backend Development:
The backend was implemented using Python Flask. It handles user authentication, manages API routes, processes data, connects to the database, and communicates with the machine learning module.

5. Database Integration:
SQLite database was used to store user credentials, wellness records, and prediction data. This ensures that all user information and analytics can be accessed and managed efficiently.

6. Machine Learning Model Development:
A machine learning model was created using the Scikit-learn library. The model analyzes data such as sleep hours, work hours, mood rating, and stress level to predict productivity and wellness patterns.

7. Machine Learning Integration:
The trained ML model was integrated into the Flask backend so that predictions can be generated in real-time when users input their wellness data.

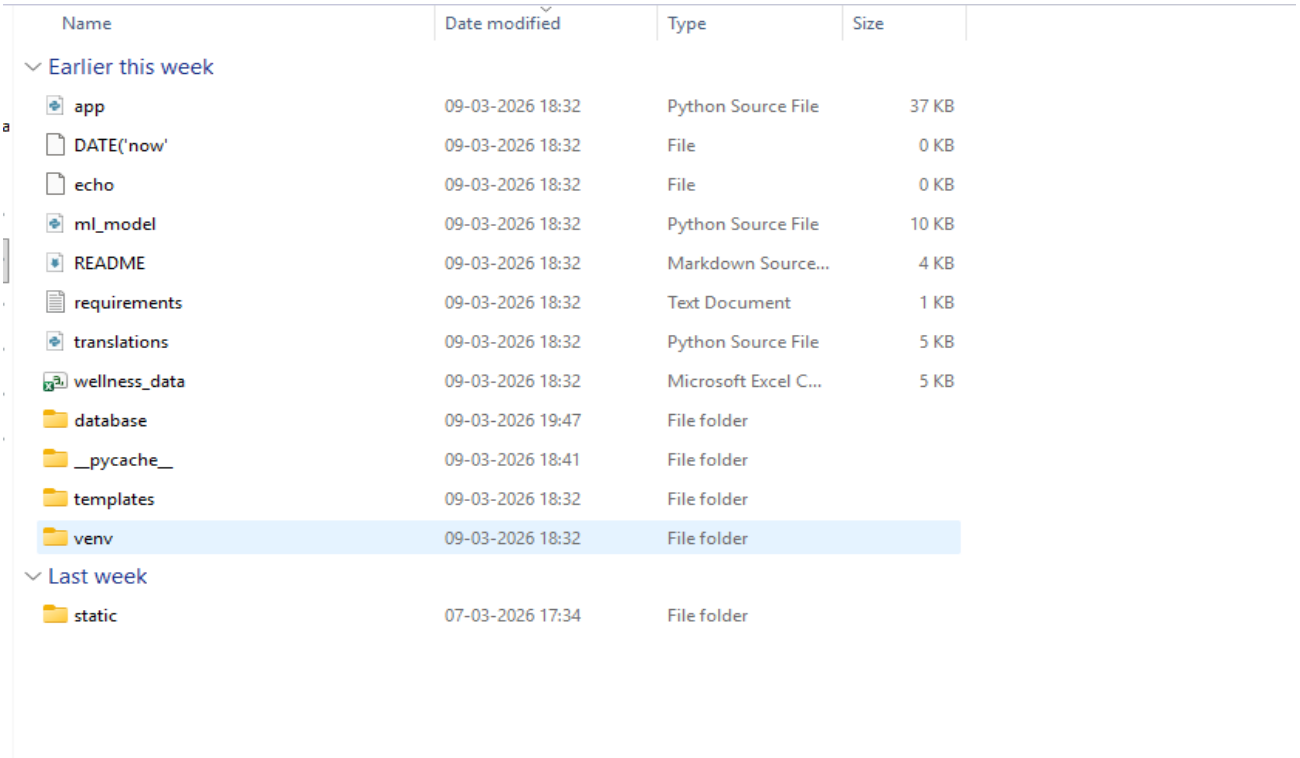
8. Dashboard Development:
An interactive dashboard was created using Chart.js to visualize wellness data through graphs and charts. This helps users easily understand their wellness trends and performance.

9. Chatbot Integration:
An AI chatbot module was added to allow users to interact with the system. The chatbot provides wellness advice and guidance based on user inputs and prediction results.

10. Admin Panel and Testing:
An admin panel was implemented to monitor users and manage system activities. Finally, the entire system was tested to ensure that all modules work correctly and integrate smoothly.

6. Snapshots / Screenshots

Opening the project folder containing all the main files such as app.py, templates, static and ml_model.py



Name	Date modified	Type	Size
Earlier this week			
app	09-03-2026 18:32	Python Source File	37 KB
DATE('now'	09-03-2026 18:32	File	0 KB
echo	09-03-2026 18:32	File	0 KB
ml_model	09-03-2026 18:32	Python Source File	10 KB
README	09-03-2026 18:32	Markdown Source...	4 KB
requirements	09-03-2026 18:32	Text Document	1 KB
translations	09-03-2026 18:32	Python Source File	5 KB
wellness_data	09-03-2026 18:32	Microsoft Excel C...	5 KB
database	09-03-2026 19:47	File folder	
__pycache__	09-03-2026 18:41	File folder	
templates	09-03-2026 18:32	File folder	
venv	09-03-2026 18:32	File folder	
Last week			
static	07-03-2026 17:34	File folder	

1. ***Opening the app.py file in VS Code to view the main Flask application code***

```
WELLBOT_ENHANCED
wellbot_enhanced
> __pycache__
> database
> static
> templates
venv
> Include
> Lib
.gitignore
pyvenv.cfg
app.py
DATE('now'
echo
ml_model.py
README.md
requirements.txt
translations.py
wellness_data.csv
```

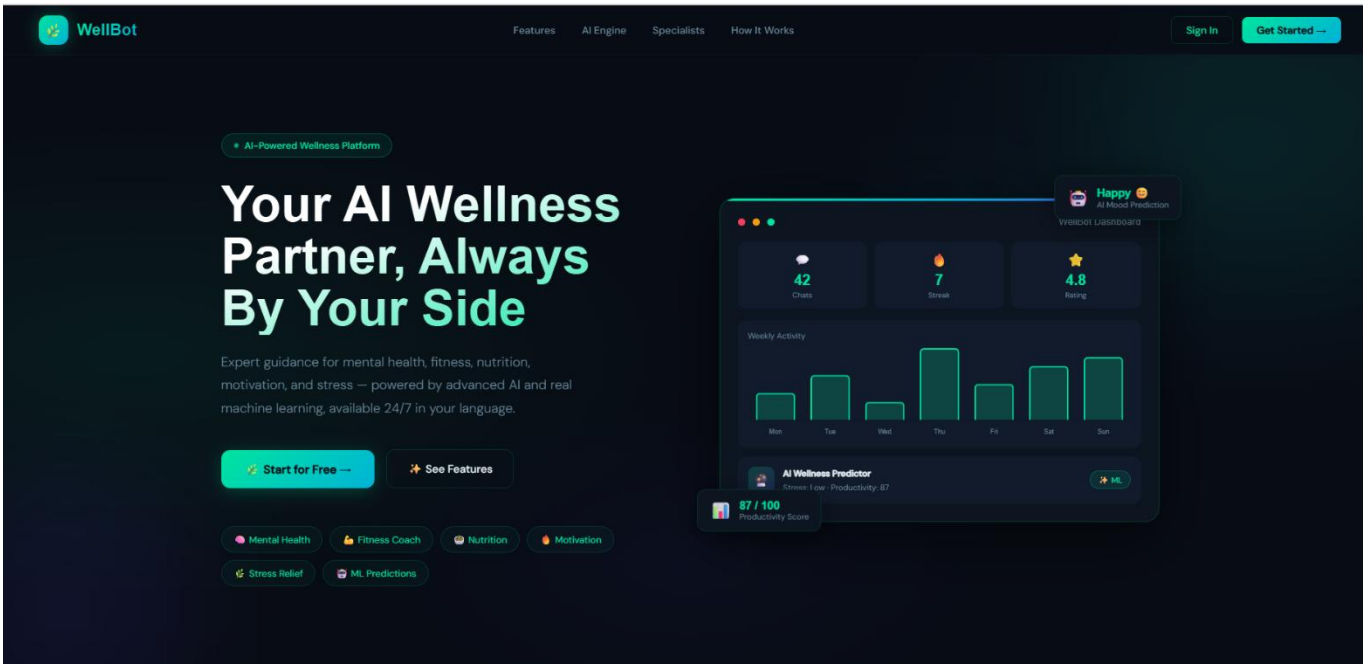
```
wellbot_enhanced > app.py > ...
import os
import json
import sqlite3
from datetime import datetime, timedelta
from flask import Flask, render_template, request, jsonify, session, redirect, url_for, send_file
from werkzeug.security import generate_password_hash, check_password_hash
from authlib.integrations.flask_client import OAuth
from translations import TRANSLATIONS
from groq import Groq
import random
import io
client = Groq(api_key="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")

# — ML MODEL INTEGRATION ————————————————————————————————————————
# Import our custom machine learning module
try:
    from ml_model import predict_wellness, get_models
    # Pre-train models when Flask starts so first request is fast
    get_models()
    ML_AVAILABLE = True
    print("[WellBot] ML models loaded successfully!")
except Exception as ml_err:                    (variable) ml_err: Exception
    print(f"[WellBot] ML models not available: {ml_err}")
    ML_AVAILABLE = False

app = Flask(__name__)
oauth = OAuth(app)

google = oauth.register(
    name='google',
    client_id='258260405503-ouhvu3edvkc244evjga84jvc120cafao.apps.googleusercontent.com',
    client_secret='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',
    server_metadata_url='https://accounts.google.com/.well-known/openid-configuration',
    client_kwargs={
```

1. ***Running the app.py in the terminal of VS code.***

```
PS C:\Users\Rashi\Downloads\wellbot_enhanced> & C:/Users/Rashi/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/Rashi/Downloads/wellbot_enhanced/wellbot_enhanced/app.py
[ML] Mood model accuracy: 100.0%
[ML] Stress model accuracy: 100.0%
[ML] Productivity model R² score: 0.993
[ML] All models trained successfully!
[WellBot] ML models loaded successfully!
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
[ML] Mood model accuracy: 100.0%
[ML] Stress model accuracy: 100.0%
```

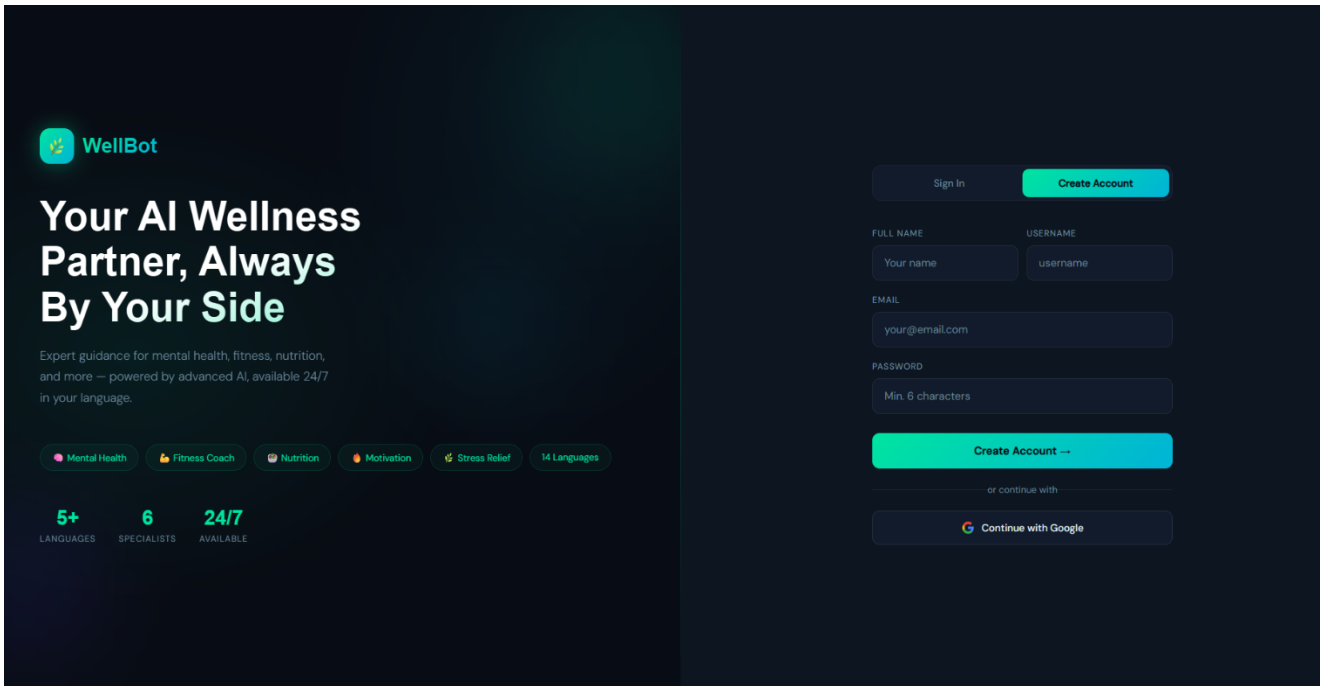
2. ***Copying or clicking the local server link provided by Flask (http://127.0.0.1:8000).***

The home page of the WellBot Global Wellness Assistant Chatbot appears on the browser.

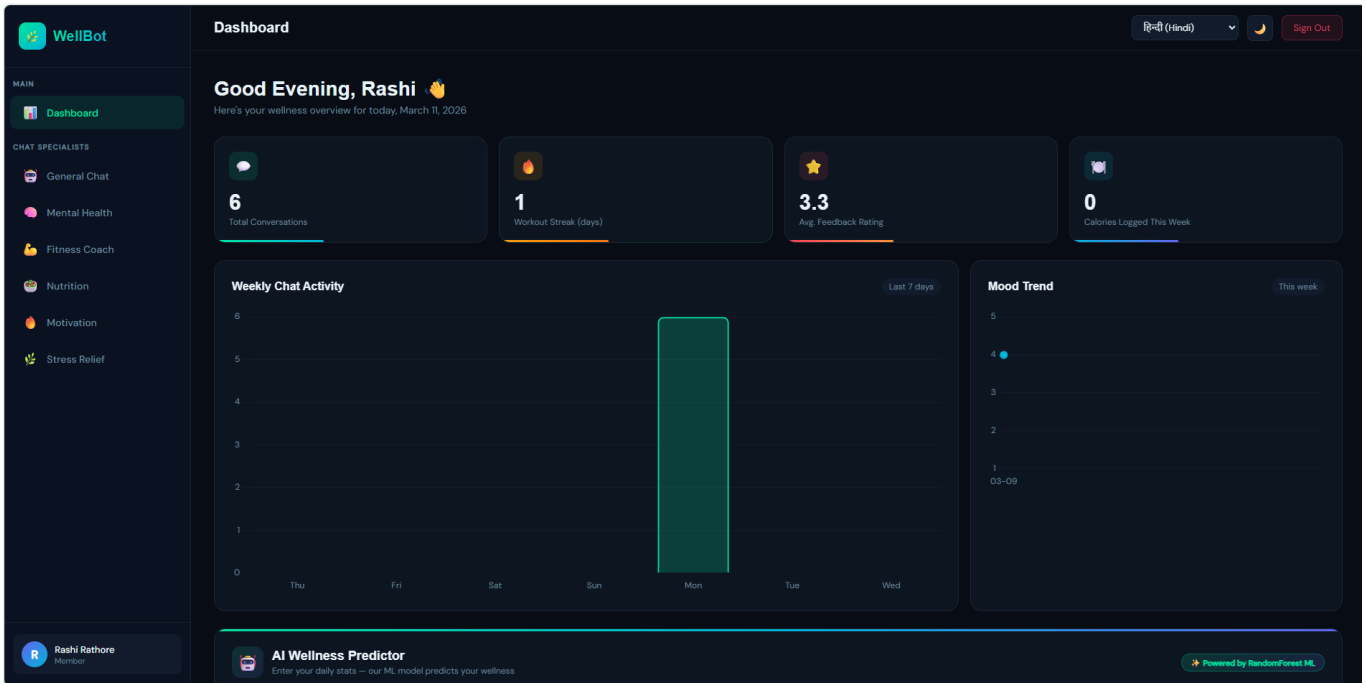


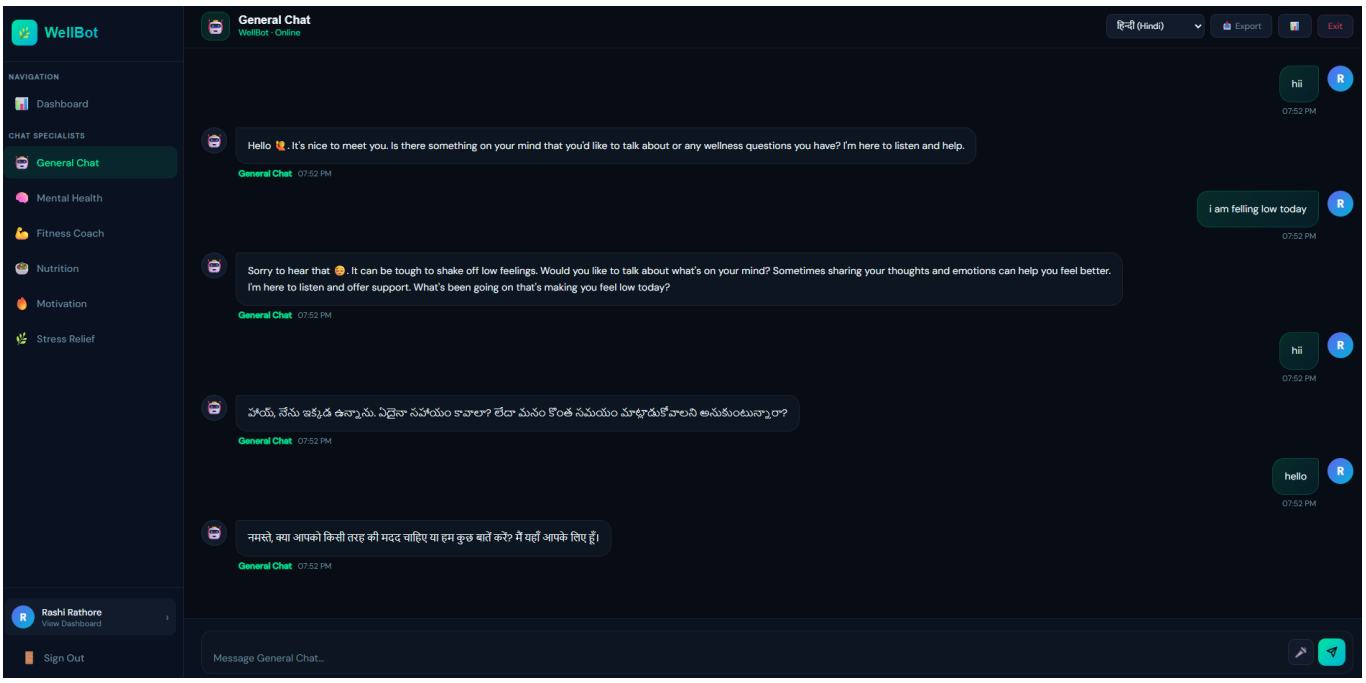
LANDING PAGE

3. **Next page includes log in and also create a account for the new user or you can start your account with google also**



DASHBOARD PAGE





Chatting with different coaches is also available in the application

7. Challenges Faced

During development of the wellness chatbot , several technical and operational challenges were encountered:

1. **Integrating Machine Learning with the Web Application:**
   One of the main challenges was connecting the machine learning model with the Flask backend so that predictions could be generated dynamically based on user inputs.
2. **Designing a User-Friendly Dashboard:**
   Creating an interactive and visually clear dashboard that displays wellness analytics in an understandable format required careful UI design and data visualization planning.
3. **Handling Data Processing and Predictions:**
   Preparing the dataset and ensuring accurate predictions for mood, stress, and productivity using the machine learning model was a complex task.
4. **Database Management:**
   Managing user data securely and maintaining proper database connectivity with SQLite while storing wellness records required careful implementation.
5. **Chatbot Integration:**
   Integrating the chatbot system and ensuring it responds correctly to user queries related to wellness and mental health was another challenge.
6. **Frontend and Backend Communication:**
   Ensuring smooth communication between the frontend interface and Flask backend through APIs required debugging and testing.
7. **System Testing and Debugging:**
   During development, several issues such as server errors, data handling problems, and interface bugs were encountered and needed to be resolved through testing.
8. **Ensuring Smooth User Experience:**
   Making sure that all modules such as login, dashboard, chatbot, and predictions worked seamlessly together was a significant challenge during the final integration phase.

8. Learnings & Skills Acquired

1. **Python Programming:**
   Understanding Python was essential for developing the backend logic of the application and implementing machine learning algorithms.
2. **Flask Framework:**
   Knowledge of Flask helped in building the web application backend, managing routes, handling requests, and integrating different modules of the system.
3. **Frontend Development:**
   Skills in HTML, CSS, and JavaScript were required to design responsive and user-friendly interfaces such as the landing page, login page, dashboard, and chatbot page.

4. **Database Management:**
   Basic knowledge of SQLite was required to store and manage user information, wellness records, and system data efficiently.
5. **Machine Learning Concepts:**
   Understanding machine learning concepts such as data preprocessing, model training, and prediction was necessary to build the wellness prediction model.
6. **Scikit-learn Library:**
   The Scikit-learn library was used to develop and train machine learning models for predicting mood, stress levels, and productivity.
7. **Data Visualization:**
   Knowledge of visualization tools like Chart.js helped in displaying wellness analytics through charts and graphs on the dashboard.
8. **Problem Solving and Debugging:**
   Strong debugging and problem-solving skills were needed to fix errors, integrate different components, and ensure smooth functionality of the system.

9. Testimonials from team

Team Member 1:

The project helped in improving analytical thinking and practical knowledge in web development, machine learning integration, and data visualization while working on a real-world wellness application."

Team Member 2:

"Working on this project strengthened problem-solving abilities through tasks such as backend development, data processing, and implementing machine learning models for wellness prediction."

Team Member 3:

"The development of the WellBot platform enhanced technical skills in Python, Flask, database management, and AI-based system design while providing experience in building real-world applications."

10. Conclusion

The **WellBot: Global Wellness Assistant Chatbot** project successfully demonstrates the integration of web development and machine learning to create an intelligent wellness monitoring system. The platform allows users to track wellness-related data such as sleep, work hours, and mood, while providing predictions and insights through a machine learning model. The system also includes an interactive dashboard, data visualization, and an AI chatbot to support users in understanding their wellness patterns. Through this project, practical knowledge of Python, Flask, database management, and machine learning was applied to build a real-world application. Overall, the project highlights how technology can be used to promote better wellness awareness and support healthier lifestyle habits.

11. Acknowledgements

We express our sincere gratitude to **Infosys Springboard** for providing an excellent learning environment, industry-oriented curriculum support, and access to valuable resources that helped in the successful development of the **WellBot Global Wellness Assistant Chatbot** project.

We extend our heartfelt thanks to **Ms. Sangeetha, our mentor**, for continuous guidance, constructive feedback, and encouragement throughout the project development process. The insights and suggestions provided played a significant role in improving our understanding and refining the implementation of the system.